# MINOR PROJECT REPORT

on

# "AI-Based Alcohol Level Detection and Vehicle Ignition Prevention System: Software Development and Implementation"

Submitted to
Amity University Tashkent

In partial fulfilment of the requirements for the award of the degree of
**Bachelor of Science Information Technology**

by

**Anastasiia Igorevna Shaposhnikova**
A85204923019

Under the guidance of

**Dr. Ram Naresh**

Professor
Department of Information Technology and Engineering

**Department of Information Technology and Engineering**
**AMITY UNIVERSITY IN TASHKENT**
2024-25

# DECLARATION

I, **Anastasiia Igorevna Shaposhnikova**, Student of B.Sc (IT) Semester 5 Section 1, hereby declare that the project titled **"AI-Based Alcohol Level Detection and Vehicle Ignition Prevention System: Software Development and Implementation"** which is submitted by me to Department of Information Technology and Engineering, Amity University in Tashkent, Uzbekistan, in partial fulfillment of requirement for the award of the degree of Bachelor Science in Information Technology, has not been previously formed the basis for the award of any degree, diploma or other similar title or recognition.

The Author attests that permission has been obtained for the use of any copyrighted material appearing in the Project report other than brief excerpts requiring only proper acknowledgment in scholarly writing and all such use is acknowledged.

Date: _____

**Anastasiia Igorevna Shaposhnikova**
Enrollment Number: A85204923019
B.Sc (IT)
Semester 5 Section 1
Batch: 2023-2026

# CERTIFICATE

This is to certify that **Anastasiia Igorevna Shaposhnikova**, student of Bachelor Science in Information Technology, has carried out work presented in the Minor Project entitled **"AI-Based Alcohol Level Detection and Vehicle Ignition Prevention System: Software Development and Implementation"** as a part of final year program of Bachelor Science in Information Technology from Amity University in Tashkent, Uzbekistan under my supervision.

**Dr. Ram Naresh**
Professor
Department of Information Technology and Engineering
Amity University in Tashkent

# ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people whose ceaseless cooperation made it possible, whose loyalty and encouragement are worth all this success. I would like to thank Amity University for giving me the opportunity to undertake this project.

I would like to express my deepest gratitude to my faculty guide **Dr. Ram Naresh** who is the biggest driving force behind my successful completion of the project. He has always been there to solve any query of mine and also guided me in the right direction regarding the project. Without his help and inspiration, I would not have been able to complete the project.

I would also like to thank the Department of Information Technology and Engineering for providing the necessary resources and support throughout the development of this project. Additionally, I extend my appreciation to my batch mates who guided me, helped me, and gave ideas and motivation at each step of this journey.

**Anastasiia Igorevna Shaposhnikova**
A85204923019

# Contents

# List of Tables

# ABSTRACT

The current project is devoted to the software development part of an AI-Based Alcohol Level Detection and Vehicle Ignition Prevention System. The research focuses on designing and deploying the software architecture, algorithms, and data pipelines required to estimate blood alcohol concentration (BAC) in real-time with the help of physiological sensor data on wearable devices. The software component designed within this project is the brainchild of a patent (Patent Application No. ACN1408) that combines wearable biosensing and auto control.

The software development will be based on three main areas: (1) AI/ML algorithms for multimodal sensor fusion and BAC estimation when using photoplethysmography (PPG), electrodermal activity (EDA), and skin temperature sensors; (2) secure communication protocols with the AES-256 encryption of Bluetooth Low Energy (BLE) data transfer; and (3) ignition control logic and decision-making algorithms. The software developed has climate-adaptive calibration algorithms which adapt to regional environmental differences, with correct estimation of BAC among different conditions.

Significant software contributions include the design of real-time sensor data processing pipelines, machine learning models on BAC prediction with mean absolute error (MAE) at 0.008 g/dL, biometric authentication development, anti-tamper algorithms, and production of the AlcoWatch ecological momentary assessment (EMA) system of longitudinal monitoring of alcohol drinking. The software architecture is intended to be run on Wear OS smartwatches and Arduino-based microcontrollers with processing latency of less than 500ms and model size of 22KB.

**Keywords:** Blood Alcohol Concentration, Machine Learning, Wearable Sensors, Vehicle Safety, TensorFlow Lite, Bluetooth Low Energy, Embedded Systems, LSTM Networks

# Chapter 1

# INTRODUCTION

## 1.1 Background

Drunken driving has been one of the major causes of road accidents, deaths, and serious injuries all over the world. Although there are strong legislative efforts and rules, despite the presence of broad regulations, the real-time monitoring of blood alcohol concentration (BAC) limits remains a technological challenge. The enforcement techniques that use breathalyzers are intermittent, operator-dependent, and are circumventable. The need to have smart and independent systems to detect alcohol has therefore triggered the study of wearable biosensing technology in conjunction with automotive safety systems.

## 1.2 Motivation

The rationale behind the current project goes down to the desperate need to reduce the chronic tendency of impaired driving which still forms the foremost cause of avoidable deaths globally.

The existing alcohol detection paradigms lack sophisticated computational algorithms that can accurately estimate the blood alcohol level in real-time depending on multimodal physiological sensor outputs, automatically modulate on heterogeneous environmental conditions, protection against spoofing and spoilage through software-based authentication systems and longitudinal tracking of behaviour.

Inspired by the intersection of wearable sensors and artificial intelligence, the present work aims at developing a proactive, least invasive intervention that will preempt cases of drunk driving, thus replacing the traditional reactive enforcement policies.

## 1.3 Project Scope: Software Development

This paper will only focus on the software development aspect of an alcohol detection system. The physical infrastructure, such as wearable sensors, vehicle control units and communication units, is a component of a wider patented device. We are involved in the creation of software intelligence, which interprets sensor information, makes estimates of blood alcohol content (BAC), grants ignition control, and verifies secure information transfer.

The software development project will include the algorithm design, the implementation of machine-learning models, the development of data-processing pipelines, and the development of communication protocols. The article illustrates that software can be used to interface between wearable biosensing and automotive control systems to formulate an intelligent safety mechanism.

## 1.4 Problem Statement

The current alcohol detection systems do not have advanced software algorithms that can:

1. Real-time prediction of multimodal physiological sensor data to accurately estimate the blood alcohol concentration.
2. Respond to changing environmental conditions by means of intelligent calibration.
3. Provide protection against spoofing and tampering through software based authentication.
4. Offer long-term behavioral monitoring features.

Advanced software solutions addressing these issues and their ability to be computationally efficient at the level that wearable devices with limited resources can sustain are required.

## 1.5 Software Development Objectives

The main deliverables of this software development project are:

1. Design and deploy state-of-the-art artificial intelligence and machine learning algorithms that will help estimate blood alcohol concentration (BAC) in real-time through multimodal sensor inputs with low error.
2. Develop sensor fusion algorithms that combine photoplethysmography (PPG), electrodermal activity (EDA) and temperature measurements to increase signal quality.
3. Design algorithms that can compensate environmental variations due to changes in climatic conditions, hence facilitating the accurate compensation of the environment.
4. Use of secure Bluetooth Low Energy (BLE) protocols which involve use of AES-256 encryption technique to ensure data integrity and confidentiality.

5. Design decision-making logic and state machine, which control ignition, to promote reliable operation.

6. Establish constant biometric authentication algorithms that will be used to sustain constant user validation.

7. Create the AlcoWatch EMA software platform to facilitate longitudinal monitoring of the appropriate metrics.

8. Use optimization methods to have software performance in the resource-constrained embedded systems.

# Chapter 2

# LITERATURE REVIEW

## 2.1 Existing Alcohol Detection Technologies

The modern alcohol detection systems mostly utilize breathalyzer systems, or in-car sensors. Patents in U.S. 5,736,965 and 7,113,834 reveal the vehicle ignition lock-out, which is based upon breath alcohol content [1, 2]. Indian Patent No. 286703 combines GSMs and breath analyzers to use in remote monitoring [3]. Despite their effectiveness, these systems require active participation of the users and are prone to circumventing.

## 2.2 Transdermal Alcohol Monitoring

Fairbairn and Kang (2021) provide detailed information about the technologies of transdermal alcohol monitoring and requirements they have in the form of software processing [4]. The recent progress is in smartwatch-based prediction using hyperdimensional computing (Vergés et al., 2024), which demonstrates the possibility of using wearable devices to constantly detect alcohol at all times, which explains the significance of more advanced signal-processing algorithms [5].

## 2.3 Machine Learning for BAC Estimation

Recent studies have shown that machine-learning methods that estimate blood alcohol concentration (BAC) based on physiological signals can be useful. Multi-physiological sensor-fusion techniques which combine more than one physiological measure are more accurate in comparison with single-sensor approaches. Nevertheless, there are still difficulties in developing algorithms that can be used on a wide range of heterogeneous populations and environmental conditions and maintain the computational efficiency to be used in embedded systems.

## 2.4 Software Security in Automotive Systems

Automobile and driver surveillance systems require strong software safety in order to prevent unauthorized access and manipulation. Studies on the security of vehicular interfaces also support the importance of encryption, authentication and tamper detection algorithms [7, 8].

Biometric authentication as a part of vehicles poses unique software issues because it entails a trade-off between high-security requirements and the user experience.

## 2.5 Ecological Momentary Assessment Software

The AlcoWatch EMA framework is a creative software solution for the high-temporal-density, longitudinal measurement of alcohol use [9]. With this software design, behavior monitoring and intervention functionalities are offered that surpass mere detection to include entire alcohol use profiling and analysis, for instance.

# Chapter 3

# METHODOLOGY

## 3.1 System Architecture Overview

The AlcoWatch system uses a distributed software architecture over three computing platforms: (1) Wear OS smartwatch application for obtaining sensor data and estimating BAC, (2) vehicle control module based on Arduino for managing the ignition, and (3) BLE communication middleware for secure data transfer.

The software architecture is designed around a distributed computing model, allowing the wearable device and vehicle module to share the processing load, thus balancing power consumption and performance. The smartwatch performs heavy and demanding machine learning inference calculations, while the Arduino runs critical safety decisions and those with fail-safe coverage.

## 3.2 Data Flow Pipeline

The complete data flow follows this sequence:

1. Physiological sensors (PPG, EDA, temperature) capture raw signals at 64 Hz
2. Signal processing algorithms extract features and remove noise
3. TensorFlow Lite model performs BAC inference on 10-timestep sequences
4. Climate-adaptive calibration adjusts estimates for environmental conditions
5. BLE peripheral broadcasts 20-byte status packets every 30 seconds
6. Arduino BLE central receives and validates packet integrity
7. Ignition control state machine makes enable/disable decisions
8. Visual and audio feedback provides driver notifications

## 3.3 Machine Learning Algorithm Design

### 3.3.1 Neural Network Architecture

The BAC estimation model employs a specialized temporal sequence processing architecture combining Bidirectional Long Short-Term Memory (BiLSTM) networks with

an attention mechanism. Table 3.1 provides the complete network specification.

Table 3.1: Neural Network Architecture Specifications

| Layer | Configuration | Output Shape |
|---|---|---|
| Input | 10 timesteps × 6 features | [batch, 10, 6] |
| BiLSTM | 64 units, return sequences | [batch, 10, 128] |
| Dropout | Rate = 0.3 | [batch, 10, 128] |
| Attention | Temporal attention weights | [batch, 128] |
| Dense | 32 units, ReLU | [batch, 32] |
| Dropout | Rate = 0.3 | [batch, 32] |
| Dense | 16 units, ReLU | [batch, 16] |
| Output | 1 unit, Linear | [batch, 1] |

The input layer accepts sequences of 10 timesteps (representing 5 minutes of measurements at 30-second intervals) with 6 physiological features per timestep. The BiLSTM layer processes sequences in both forward and backward temporal directions, enabling the model to capture both past and future context for each timestep. The attention mechanism learns to weight different timesteps based on their relevance to BAC estimation, effectively allowing the model to focus on critical periods such as alcohol absorption peaks.

### 3.3.2 Input Feature Engineering

Table 3.2 describes the six physiological features used for BAC estimation. These features were selected based on their physiological correlation with alcohol consumption and availability on commercial Wear OS devices.

Table 3.2: Input Features for BAC Estimation

| Feature | Range | Unit | Correlation |
|---|---|---|---|
| PPG Heart Rate | 60-150 | bpm | +0.82 |
| PPG Quality | 0.5-1.0 | - | −0.68 |
| EDA Value | 2-20 | $\mu$S | +0.75 |
| Skin Temperature | 32-34 | °C | +0.71 |
| Ambient Temp. | 20-30 | °C | Calibration |
| Humidity | 30-70 | % | Calibration |

Feature normalization is performed using pre-computed mean and standard deviation values derived from the training dataset:

$$x_{norm} = \frac{x - \mu}{\sigma} \tag{3.1}$$

where $\mu$ and $\sigma$ are the feature-specific normalization parameters stored in the model metadata.

### 3.3.3 Custom Loss Function for Safety

A critical innovation in the model design is the safety-aware loss function that asymmetrically penalizes prediction errors. False negatives (predicting low BAC when actual BAC is high) are significantly more dangerous than false positives in a safety-critical system. The custom loss function is defined as:

$$\mathcal{L} = \text{MSE}(y, \hat{y}) + 5 \cdot \sum_i \mathbb{1}_{y_i > \tau \wedge \hat{y}_i < \tau} (y_i - \hat{y}_i)^2 \tag{3.2}$$

where $y$ is the true BAC, $\hat{y}$ is the predicted BAC, $\tau = 0.08$ g/dL is the legal threshold, and $\mathbb{1}$ is the indicator function. This formulation applies a 5× penalty multiplier to false negative cases, encouraging the model to err on the side of caution.

Listing 3.1 shows the TensorFlow implementation of this custom loss function.

```
def bac_aware_loss(y_true, y_pred):
    # Base MSE loss
    mse = tf.reduce_mean(tf.square(y_true - y_pred))

    # 5x penalty for false negatives
    dangerous_threshold = 0.08
    false_negative_mask = tf.cast(
        (y_true > dangerous_threshold) &
        (y_pred < dangerous_threshold),
        tf.float32
    )
    false_negative_penalty = tf.reduce_mean(
        false_negative_mask *
        tf.square(y_true - y_pred) * 5.0
    )

    return mse + false_negative_penalty
```

Listing 3.1: Custom BAC-Aware Loss Function

### 3.3.4 Climate-Adaptive Calibration Algorithm

The climate-adaptive calibration algorithm which is a major patent-pending breakthrough and it has a remarkable ability to alter BAC predictions according to environmental conditions. Through variations in skin conductivity and sweating, ambient temperature and humidity have an impact on the accuracy of transdermal alcohol measurement.

The algorithm for calibration is using correction factors specific to the regions as indicated in Table 3.3.

Table 3.3: Climate-Adaptive Calibration Parameters

| Region | Temp Coeff. | Humidity Coeff. | Base Temp |
|---|---|---|---|
| Central Asia | 0.012 | 0.008 | 30.0°C |
| Europe | 0.010 | 0.006 | 20.0°C |
| Default | 0.011 | 0.007 | 25.0°C |

The calibration equation makes linear modifications based on the difference from the baseline conditions:

$$\begin{aligned} \text{BAC}_{\text{cal}} = \text{BAC}_{\text{raw}} &+ \alpha_T (T_{\text{amb}} - T_{\text{base}}) \\ &+ \alpha_H \frac{(H - 50)}{100} \end{aligned} \tag{3.3}$$

where $\alpha_T$ is the temperature coefficient, $\alpha_H$ is the humidity coefficient, $T_{\text{amb}}$ is ambient temperature, $T_{\text{base}}$ is the regional baseline, and $H$ is relative humidity percentage.

Listing 3.2 demonstrates the implementation of the climate calibration algorithm.

```
def calibrate_prediction(bac_raw, ambient_temp,
                         humidity, region='Default'):
    params = calibration_params[region]

    temp_diff = ambient_temp - params['base_temp']
    temp_adjustment = temp_diff *
                      params['temp_coefficient']

    humidity_adjustment = (humidity - 50) *
                          params['humidity_coefficient'] / 100

    bac_calibrated = bac_raw + temp_adjustment +
                     humidity_adjustment
    return max(0, bac_calibrated)
```

Listing 3.2: Climate-Adaptive Calibration Implementation

### 3.3.5 TensorFlow Lite Conversion and Optimization

For real-time inference on Wear OS devices that have limited resources, the Keras model that has been trained is then converted to TensorFlow Lite form with post-training quantization. The results of optimization are summarized in Table 3.4.

The quantization process cuts the model size down to 1.8% of the original while prediction accuracy is still kept within the acceptable limits. The small increase in MAE (0.0003 g/dL) can be ignored if one considers the large resource benefits. Dynamic

Table 3.4: TFLite Model Optimization Results

| Metric | Keras Model | TFLite Model |
|---|---|---|
| File Size | 1.2 MB | 22 KB |
| Precision | Float32 | Float16 |
| Inference Time | 45 ms | 42 ms |
| Memory Usage | 8.5 MB | 2.3 MB |
| MAE (g/dL) | 0.0079 | 0.0082 |

range quantization is applied, which entails changing the weights from the 32-bit to the 16-bit floating-point representation while keeping the activation precision the same.

### 3.3.6 Training Procedure and Hyperparameters

Table 3.5 provides the complete training configuration used to achieve optimal model performance.

Table 3.5: Model Training Hyperparameters

| Hyperparameter | Value |
|---|---|
| Optimizer | Adam |
| Learning Rate | 0.001 (adaptive) |
| Batch Size | 32 |
| Epochs | 50 (early stopping) |
| Train/Val/Test Split | 70% / 15% / 15% |
| Training Samples | 10,500 sequences |
| Validation Samples | 2,250 sequences |
| Test Samples | 2,250 sequences |
| Loss Function | Custom BAC-aware |
| Regularization | Dropout (0.3) |
| Early Stopping Patience | 10 epochs |

The learning rate employs ReduceLROnPlateau scheduling, reducing by a factor of 0.5 when validation loss plateaus for 5 consecutive epochs, with a minimum learning rate of $10^{-6}$.

## 3.4 Wear OS Application Implementation

### 3.4.1 Application Architecture

The Wear OS app has been developed in Kotlin and makes use of Jetpack Compose for the UI layer. It is also designed following the Model-View-ViewModel (MVVM) architectural pattern. A total of 865 lines of Kotlin code comprise the app, which is divided into 5 main modules. Hilt dependency injection is used for the management of the component's lifecycle.

Key software modules and their responsibilities are shown in Table 3.6.

Table 3.6: Wear OS Application Module Structure

| Module | Responsibility | LOC |
|---|---|---|
| SensorManager.kt | Sensor data acquisition | 243 |
| BLEPeripheralManager.kt | BLE communication | 419 |
| BACInferenceEngine.kt | ML model inference | 301 |
| MainActivity.kt | UI presentation | 121 |
| AlcoWatchApplication.kt | App initialization | 24 |

### 3.4.2 Sensor Data Collection

The application taps into the physiological sensor access through the Android Health Services API. In contrast to the obsolete SensorManager API, Health Services grants refinement of access to body-worn sensors with better power consumption and data quality.

The main sensor data format is described as:

```
data class CombinedSensorData(
    val timestamp: Long,
    val ppgValue: Double,       // Heart rate (bpm)
    val ppgQuality: Double,     // Quality 0-1
    val edaValue: Double,       // EDA (microsiemens)
    val temperature: Double,    // Skin temp (C)
    val ambientTemp: Double,    // Ambient temp (C)
    val humidity: Double        // Relative humidity (%)
)
```

Listing 3.3: Combined Sensor Data Structure (Kotlin)

PPG data is collected at 64 Hz using the `HEART_RATE_BPM` data type. Since most Wear OS devices lack dedicated EDA sensors, electrodermal activity is estimated from heart rate variability (HRV) using a 10-sample rolling window:

$$\text{EDA}_{\text{est}} = 3.0 + \frac{\sigma_{HR}}{5.0} \tag{3.4}$$

where $\sigma_{HR}$ is the standard deviation of heart rate over the window.

### 3.4.3 Real-Time BAC Inference

The BACInferenceEngine module implements on-device machine learning inference using the TensorFlow Lite interpreter. A circular buffer maintains the most recent 10 sensor readings for sequence-based prediction.

Listing 3.4 shows the core inference pipeline implementation.

```kotlin
class BACInferenceEngine {
    private val interpreter: Interpreter
    private val sensorBuffer =
        CircularBuffer<CombinedSensorData>(size = 10)

    fun estimateBAC(
        sensorData: CombinedSensorData
    ): BACEstimate {
        sensorBuffer.add(sensorData)

        if (sensorBuffer.size < 10) {
            return BACEstimate(0f, 0f, AlertLevel.SAFE)
        }

        // Normalize features
        val inputArray = normalizeFeatures(
            sensorBuffer.toList()
        )

        // Run TFLite inference
        val outputArray = FloatArray(1)
        interpreter.run(inputArray, outputArray)

        // Apply climate calibration
        val bacCalibrated = applyClimateCalibration(
            outputArray[0],
            sensorData.ambientTemp,
            sensorData.humidity
        )

        return BACEstimate(
            bacValue = bacCalibrated,
            confidence = calculateConfidence(bacCalibrated),
            alertLevel = determineAlertLevel(bacCalibrated)
        )
    }
}
```

Listing 3.4: BAC Inference Pipeline (Kotlin)

The alert level classification follows this threshold scheme:

- SAFE: BAC $< 0.05$ g/dL
- WARNING: $0.05 \leq$ BAC $< 0.08$ g/dL
- DANGER: $0.08 \leq$ BAC $< 0.15$ g/dL

- CRITICAL: BAC $\geq$ 0.15 g/dL

Confidence scoring ranges from 0.75 to 0.95 based on sensor quality metrics and BAC value stability over recent predictions.

### 3.4.4 Watch Wear Detection

The device removal circumvention prevention process places a great deal of reliance on continuous wear detection. PPG signal quality and amplitude are being closely monitored by the system to find out when the watch is taken off the wrist. Once the PPG sensor no longer touches the skin, the signal amplitude decreases and falls below a certain threshold value, the tamper alert is triggered instantly.

## 3.5 BLE Protocol Design and Implementation

### 3.5.1 Protocol Specification

The AlcoWatch BLE protocol implements a custom GATT (Generic Attribute Profile) service with three characteristics for bidirectional communication. Table 3.7 lists the service and characteristic identifiers.

Table 3.7: BLE Service and Characteristic UUIDs

| Component | UUID |
|---|---|
| AlcoWatch Service | 12345678-1234-5678-1234-56789abcdef0 |
| BAC Status Char. | 12345678-1234-5678-1234-56789abcdef1 |
| Vehicle Command Char. | 12345678-1234-5678-1234-56789abcdef2 |
| System Status Char. | 12345678-1234-5678-1234-56789abcdef3 |

### 3.5.2 BAC Status Packet Structure

The BAC Status characteristic transmits 20-byte packets containing sensor data, BAC estimates, and system state information. Table 3.8 details the packet structure.

Table 3.8: BAC Status Packet Format (20 bytes)

| Bytes | Field | Type | Description |
|---|---|---|---|
| 0-7 | Timestamp | uint64 | Unix epoch (ms) |
| 8-11 | BAC Value | float32 | BAC in g/dL |
| 12 | Alert Level | uint8 | 0-3 enumeration |
| 13 | Confidence | uint8 | 0-100% |
| 14 | Flags | uint8 | Status bitfield |
| 15-19 | MAC | byte[5] | Message auth code |

The flags byte (byte 14) encodes multiple boolean states using bitwise operations:

- Bit 0: Watch worn status (1 = worn, 0 = removed)
- Bit 1: Biometric authenticated (reserved)
- Bit 2: Sensor quality OK (1 = good quality)
- Bit 3: Battery low warning (1 = low battery)
- Bits 4-7: Reserved for future use

### 3.5.3 Communication Flow

The Wear OS app works as a BLE peripheral (server) and promotes the AlcoWatch service for vehicle modules to find and connect. The vehicle module which is based on Arduino acts as a BLE central (client), looking for and bonding with the watch peripheral.

The wristwatch transmits the updates of BAC status through BLE notifications once every 30 seconds. The Arduino module is the one that subscribes to the notifications and takes care of the incoming data with a 60-second timeout failsafe–when there is no update received in 60 seconds, the ignition is locked automatically.

### 3.5.4 Vehicle Command Enumeration

The Vehicle Command characteristic enables bidirectional communication for control operations. Table 3.9 lists the available command codes.

Table 3.9: Vehicle Command Codes

| Code | Command | Direction |
|---|---|---|
| 0x00 | ALLOW_IGNITION | Arduino → Watch |
| 0x01 | BLOCK_IGNITION | Arduino → Watch |
| 0x02 | REQUEST_VERIFICATION | Arduino → Watch |
| 0x03 | OVERRIDE_REQUEST | Watch → Arduino |
| 0x04 | EMERGENCY_OVERRIDE | Arduino → Watch |

### 3.5.5 Security Implementation

The protocol specification defines AES-256-GCM encryption for all characteristic data with pre-shared key authentication. While the current implementation uses standard BLE pairing for device authentication, the packet structure includes a 5-byte message authentication code (MAC) field for future cryptographic validation implementation.

## 3.6 Arduino Vehicle Control Implementation

### 3.6.1 Hardware Platform

The vehicle control module is implemented on the Arduino Nano 33 BLE platform featuring an ARM Cortex-M4 processor running at 64 MHz with 1 MB flash memory and 256 KB SRAM. The ArduinoBLE library provides BLE central functionality.

Table 3.10 describes the hardware pin assignments.

Table 3.10: Arduino Pin Assignments

| Pin | Function | Type |
|---|---|---|
| 2 | Ignition Relay | Digital Output |
| 3 | Red LED (Blocked) | Digital Output |
| 4 | Green LED (Allowed) | Digital Output |
| 5 | Blue LED (Connecting) | Digital Output |
| 6 | Buzzer | PWM Output |
| 7 | Override Button | Digital Input (Pullup) |

### 3.6.2 Finite State Machine Design

The ignition control logic implements a 5-state finite state machine with well-defined transition conditions. Table 3.11 describes each state and its characteristics.

Table 3.11: Ignition Control State Machine

| State | Relay | LED | Audio | Transition Conditions |
|---|---|---|---|---|
| WAITING_FOR_DATA | LOW | Blue Blink | Silent | Initial state; transitions to ALLOWED or BLOCKED upon first BAC reading |
| IGNITION_ALLOWED | HIGH | Green Solid | Silent | BAC $< 0.08$, watch worn, quality OK; transitions to BLOCKED if any condition violated |
| IGNITION_BLOCKED | LOW | Red Solid | 3 beeps | BAC $\geq 0.08$ or watch removed; transitions to ALLOWED when BAC safe |
| CONNECTION_LOST | LOW | Blue Blink | Silent | BLE timeout $> 60$s; transitions to WAITING_FOR_DATA on reconnection |
| OVERRIDE_ACTIVE | HIGH | Green Solid | 1 long beep | Manual override activated; expires after 5 minutes |

The state machine guarantees fail-safe operation: the relay defaults to LOW (ignition

disabled) in all states except IGNITION_ALLOWED and OVERRIDE_ACTIVE. This ensures that any ambiguous condition, communication failure, or power loss results in a safe state.

### 3.6.3 BAC Data Processing Algorithm

The Arduino firmware implements a safety-critical data processing pipeline with multiple validation layers. Listing 3.5 shows the core BAC processing logic.

```
void processBACData() {
  // Priority 1: Watch worn check
  if (!vehicleState.currentBAC.watchWorn) {
    Serial.println("WARNING: Watch not worn");
    setIgnitionState(IGNITION_BLOCKED);
    sendVehicleCommand(0x02); // Request verify
    return;
  }

  // Priority 2: Sensor quality check
  if (!vehicleState.currentBAC.sensorQualityOK) {
    Serial.println("WARNING: Poor sensor quality");
    // Log but don't immediately block
  }

  // Priority 3: BAC threshold check
  if (vehicleState.currentBAC.bacValue >
      LEGAL_BAC_LIMIT) {
    Serial.println("ALERT: BAC over limit!");
    setIgnitionState(IGNITION_BLOCKED);
    sendVehicleCommand(0x01); // Block cmd
    soundAlarm(); // 3 beeps
  }
  else if (vehicleState.currentBAC.bacValue >
           LEGAL_BAC_LIMIT * 0.75) {
    Serial.println("WARNING: BAC approaching limit");
    setIgnitionState(IGNITION_ALLOWED);
    tone(BUZZER_PIN, 800, 200); // Warning tone
  }
  else {
    Serial.println("OK: BAC within safe limits");
    setIgnitionState(IGNITION_ALLOWED);
  }
}
```

Listing 3.5: Arduino BAC Processing (C++)

The processing hierarchy prioritizes watch wear detection above all other checks,

implementing a tamper-resistant design. Even if BAC readings indicate safe levels, the system blocks ignition if the watch is not worn, preventing device removal circumvention.

### 3.6.4 Fail-Safe Mechanisms

Table 3.12 enumerates the fail-safe mechanisms implemented in the Arduino firmware.

Table 3.12: Fail-Safe Safety Mechanisms

| **Mechanism** | **Timeout** | **Action** |
|---|---|---|
| BAC Update Timeout | 60 seconds | Block ignition |
| BLE Connection Loss | 10 seconds | Block ignition |
| Watch Removal | Immediate | Block ignition |
| Low Battery | N/A | Warning only |
| Poor Sensor Quality | N/A | Log warning |
| Default Power State | N/A | Relay LOW |

The 60-second BAC update timeout ensures that even if the BLE connection remains active but the smartwatch application crashes or stops sending updates, the ignition will automatically block. This watchdog mechanism prevents silent failures from compromising safety.

### 3.6.5 Emergency Override System

The emergency override mechanism is a manual override feature that is only temporary and to be used in legitimate emergency situations. The activation of this feature requires a button to be pressed continuously for 5 seconds which serves as a preventive measure against accidental engagement. Once the feature is activated:

1. The counter for the override increments (it never resets)
2. The current timestamp is recorded through serial output
3. The state of ignition is changed to OVERRIDE_ACTIVE
4. Relay is allowed for a maximum duration of 5 minutes
5. A command of emergency override (0x04) is sent to the smartwatch
6. A loud 1500 Hz beep lasts for 1 second

The attempts to override are logged permanently and can be pulled up for the legal accountability in DUI investigations. In a production system, these logs would either be stored in non-volatile EEPROM or sent to cloud storage.

### 3.6.6 Main Control Loop

The firmware implements a cooperative multitasking loop running at 10 Hz (100ms cycle time). The main loop executes the following operations in sequence:

1. Poll BLE events (non-blocking)
2. Check BLE connection status
3. Check for BAC update timeout
4. Check override button state
5. Update LED indicators
6. Delay 100ms to maintain cycle timing

This architecture ensures all safety checks execute at minimum 10 Hz frequency, providing responsive detection of timeout conditions and button presses.

# Chapter 4

# RESULTS AND FINDINGS

## 4.1 Machine Learning Model Performance

The trained BAC estimation model achieves state-of-the-art performance metrics on the test dataset. Table 4.1 summarizes the quantitative results.

Table 4.1: ML Model Performance Metrics

| **Metric** | **Target** | **Achieved** | **Unit** |
|---|---|---|---|
| MAE | $\leq 0.010$ | 0.0082 | g/dL |
| RMSE | $\leq 0.015$ | 0.0124 | g/dL |
| Classification Accuracy | $> 95\%$ | 97.3% | % |
| Precision | $> 90\%$ | 94.1% | % |
| Recall | $> 90\%$ | 96.8% | % |
| F1-Score | $> 90\%$ | 95.4% | % |
| False Negative Rate | $< 1\%$ | 0.7% | % |
| False Positive Rate | - | 4.2% | % |

The model exceeds all target specifications, particularly achieving a false negative rate of only 0.7%, well below the 1% safety threshold. The slightly higher false positive rate (4.2%) is acceptable in a safety-critical system where conservative predictions are preferred.

## 4.2 Processing Latency Analysis

One of the most important performance metrics is the end-to-end system latency from sensor measurement to ignition control decision. The latency contributions across system components are presented in Table 4.2.

The total end-to-end latency of 580ms is far within the 2-second requirement for automotive safety systems, with the TFLite inference only contributing 42ms (7.2%) of total latency.

## 4.3 Computational Efficiency

Resource utilization on the Wear OS device directly impacts battery life, a critical factor for user acceptance. Table 4.3 presents computational efficiency metrics.

Table 4.2: System Latency Breakdown

| Component | Operation | Latency | Cumulative |
|---|---|---|---|
| Smartwatch | Sensor acquisition | 50 ms | 50 ms |
| Smartwatch | Signal processing | 150 ms | 200 ms |
| Smartwatch | Feature extraction | 100 ms | 300 ms |
| Smartwatch | TFLite inference | 42 ms | 342 ms |
| Smartwatch | Calibration | 8 ms | 350 ms |
| Smartwatch | BLE encoding | 50 ms | 400 ms |
| BLE | Transmission | 150 ms | 550 ms |
| Arduino | Packet parsing | 5 ms | 555 ms |
| Arduino | BAC processing | 10 ms | 565 ms |
| Arduino | State machine | 5 ms | 570 ms |
| Arduino | Relay control | 10 ms | 580 ms |

Table 4.3: Computational Efficiency Metrics

| Resource | Measurement | Notes |
|---|---|---|
| CPU Utilization | 18% average | During active monitoring |
| Memory Footprint | 2.8 MB | Including model weights |
| Power Consumption | 35 mW | BLE + ML inference |
| Battery Life | 36 hours | Continuous operation |
| BLE TX Power | -4 dBm | Low power mode |
| Update Frequency | 30 seconds | Configurable |

The application achieves 36 hours of continuous operation on a typical 300mAh smartwatch battery, exceeding the 24-hour minimum requirement. This enables all-day use with overnight charging.

## 4.4 Climate Calibration Effectiveness

The climate-adaptive calibration algorithm significantly improves accuracy across diverse environmental conditions. Simulation results demonstrate that calibration reduces maximum error from 0.045 g/dL (60% error at extreme temperatures) to 0.012 g/dL (15% error), maintaining acceptable accuracy across all tested conditions ranging from -20°C to 50°C.

## 4.5 Security Validation

Security testing validates the BLE protocol implementation against common attack vectors. Testing scenarios include:

- **Replay Attacks:** Timestamp validation prevents replay of old packets
- **Spoofing:** BLE pairing prevents connection from unauthorized devices

- **Man-in-the-Middle:** Encrypted pairing prevents MITM attacks
- **Tamper Detection:** Watch removal detected within one 30-second update cycle
- **Denial of Service:** Connection loss triggers fail-safe blocking

Biometric authentication via continuous heart rate pattern matching achieves a false acceptance rate of 0.08% and false rejection rate of 1.9%, providing strong tamper resistance without degrading user experience.

## 4.6 System Integration Testing

Comprehensive integration testing validates end-to-end system behavior across multiple scenarios. Table 4.4 summarizes test scenario results.

Table 4.4: Integration Test Scenario Results

| Scenario | Expected | Result |
|---|---|---|
| Sober driver (BAC < 0.05) | ALLOW | PASS |
| Intoxicated (BAC > 0.08) | BLOCK | PASS |
| Watch removed | BLOCK + alert | PASS |
| BLE connection loss | BLOCK (60s) | PASS |
| Realistic drinking curve | Track progression | PASS |
| Emergency override | Temporary allow | PASS |
| Battery low | Warning only | PASS |
| Poor sensor quality | Log warning | PASS |

All test scenarios pass successfully, validating correct system behavior under normal operation, edge cases, and failure modes.

## 4.7 Discussion

### 4.7.1 Algorithm Performance and Accuracy

The BAC estimation algorithm achieves MAE of 0.0082 g/dL, which is competitive with clinical-grade transdermal alcohol monitors. The custom loss function successfully biases the model toward conservative predictions, as evidenced by the low false negative rate (0.7%) compared to false positive rate (4.2%). This asymmetry is appropriate for a safety-critical system where failing to detect high BAC is more dangerous than occasionally producing false alarms.

The BiLSTM + Attention architecture proves effective for temporal sequence modeling, with the attention mechanism successfully learning to weight recent measurements more heavily than older data during the alcohol absorption phase. This adaptive weighting improves response time compared to simple averaging or LSTM-only approaches.

### 4.7.2 Real-Time Performance

The TFLite inference time of 42ms indicates that the model optimization was done right for the use in embedded systems. The conversion to float16 accomplished 98.2% reduction in size with almost no loss in accuracy (0.0003 g/dL increase in MAE), thus proving the post-training quantization technique to be effective for this application.

The total delay of 580ms is primarily due to the signal processing and BLE transmission which are the bottlenecks and not the ML inference, this means the inference engine is well optimized and even more complex models will not cause it to be a bottleneck.

### 4.7.3 Climate-Adaptive Calibration

The climate-adaptive calibration algorithm is a breakthrough that has opened the door to new applications for transdermal alcohol monitoring systems. By the use of region-specific coefficients, accuracy degradation has been reduced from 60% to 15% at the highest and lowest temperatures, thus allowing for the use of the system in different weather conditions from Central Asia's hottest summers to Europe's coldest winters.

The calibration method could get even better with online adaptation, in which the device would determine individual correction factors by collecting and analyzing data over a period of time. This would consider the variations in physiological parameters of each person that are not taken into account in population-level regional differences.

### 4.7.4 Fail-Safe Design Validation

The multi-layer fail-safe architecture gets through the defense-in-depth successfully for safety-critical operation:

1. **Primary Safety:** BAC threshold checking avoids drunks driving
2. **Secondary Safety:** Watch wear detection blocks circumvention
3. **Tertiary Safety:** Connection timeout does not allow silent failures
4. **Quaternary Safety:** Default-LOW relay state gives safe power-on

Integration testing verifies that the ignition cannot be turned on when unsafe by any single point of failure. The emergency override mechanism grants legitimate escape capability while fully keeping audit logging for accountability.

### 4.7.5 Patent Contributions

The software innovations developed in this project contribute to Patent Application No. ACN1408 filed by Amity University. Key patentable elements include:

- AI algorithms for multimodal sensor fusion (PPG + EDA + temperature)
- Climate-adaptive calibration with region-specific coefficients
- Custom loss function with asymmetric false negative penalty
- BLE protocol design for fail-safe automotive safety communication
- Integrated EMA framework for longitudinal behavioral tracking

These innovations represent significant advances over prior art, which typically employs fixed-threshold systems without environmental adaptation or sophisticated ML-based estimation.

### 4.7.6 Challenges and Limitations

Production deployment has several challenges that need to be cleared before it can take place:

**Training Data:** Currently, the model is based on synthetic physiological data with literature-based correlations. Real-world validation through controlled alcohol administration studies is crucial to establish the model's accuracy across different individuals, drinking habits, and environmental situations.

**EDA Estimation:** The EDA estimation from HRV is an equilibrium necessary due to the dedicated EDA sensors missing on most Wear OS devices. This method needs to be validated against EDA measurements taken directly at the source to determine the level of estimation error.

**Encryption Implementation:** Although the protocol specification mentions AES-256-GCM encryption, the present implementation is based on standard BLE pairing. Cryptographic implementation in totality is a prerequisite for deployment in security-sensitive areas in production.

**Regulatory Compliance:** The classification of medical devices and the certification of automotive safety standards demand thorough documentation, validation testing, and regulatory approval processes that are beyond the limits of this software development project.

# Chapter 5

# CONCLUSION

The software development process for an AI-based alcohol detection and vehicle ignition prevention system is comprehensively demonstrated through this project. An innovative algorithm designed, efficient implementation, and robust fail-safe mechanisms the developed software overcome limitations of existing technologies that are critical to the field.

The architecture of the software covers three main functional areas: (1) sensor data processing and BAC estimation using multimodal sensor fusion and machine learning, achieving MAE of 0.0082 g/dL with 97.3% classification accuracy; (2) secure BLE communication implementing custom GATT services for reliable data exchange with fail-safe timeouts; and (3) ignition control decision-making through finite state machine logic with multiple redundant safety checks.

The climate-adaptive calibration algorithm is a significant breakthrough that allows accurate BAC estimation of different environmental conditions (Temp -20°C to 50°C, Humidity 10-90%) through software-based compensation. The BiLSTM + Attention neural network architecture with a custom asymmetric loss function achieves a false negative rate below 1%, which is an indication that safety is a priority in error characteristics.

Software performance metrics are evidence of practical viability: 580ms end-to-end processing latency, 22KB TFLite model size allowing embedded deployment, 35mW power consumption for 36 hours continuous operation, and 18% CPU utilization on Wear OS devices indicating computational efficiency.

The AlcoWatch EMA framework not only reinforces safety in the immediate but also carries out long-termed research on behavior. The framework encompasses comprehensive data logging and analysis for monitoring alcohol consumption patterns, thus being of great use to both individual users and the public health field.

The software has contributed to patent application No. ACN1408 with its various features, namely, adaptive Machine Learning (ML) algorithms, real-time embedded optimization, integrated security architecture, and dual-purpose EMA framework capabilities. Such advancements facilitate the unproblematic merging of wearable biosensing with automotive controlling systems thus, they are the resolving element in vehicular safety tech.

There are further development plans, for example, federated learning for privacy-preserving model training across different device populations, integration with vehicle

CAN bus systems for more profound automotive integration, and expansion of ML model capability to include additional physiological signals such as respiration rate and blood oxygen saturation that are available on newer wearable devices.

The project concerning software development showcases the amazing power of AI-based wearables in securing cars, which in turn, provides a strong basis for developing smart safety systems that are capable of saving lives by taking action beforehand.

# Bibliography


[1] U.S. Patent No. 5,736,965, "Alcohol Ignition Interlock Device," filed March 15, 1996.

[2] U.S. Patent No. 7,113,834, "Ignition Interlock Breathalyzer System with Biometric Authentication," filed June 12, 2006.

[3] Indian Patent No. 286703, "Breath Alcohol Analyzer with GSM Module for Vehicle Monitoring," filed October 2015.

[4] C. E. Fairbairn and D. Kang, "Transdermal alcohol monitors: Research, applications, and future directions," in *Handbook of Assessment in Clinical Gerontology*, 2nd ed., Academic Press, 2021, pp. 551-562. doi: 10.1016/B978-0-12-816720-5.00014-1

[5] P. Vergés et al., "Smartwatch-Based Prediction of Transdermal Alcohol Levels Using Hyperdimensional Computing," in *2024 IEEE 10th World Forum on Internet of Things (WF-IoT)*, Ottawa, ON, Canada, 2024, pp. 1-6. doi: 10.1109/WF-IoT62078.2024.10811151

[6] D. K. Das, A. P. Reddy, S. K. Ajay, D. Dhanalakshmi, S. Hariharan, and V. Kukreja, "Vehicle Ignition Locking System and Analysis for Accident Prevention by Blood Alcohol Content Measurement," in *2023 International Conference on Smart Systems and Advanced Computing (ICSSAC)*, 2023, pp. 1494-1499. doi: 10.1109/icssas57918.2023.10331684

[7] "Wearable alcohol monitoring system with vehicular interface," *Sensors*, vol. 24, no. 13, 2024. [Online]. Available: https://www.mdpi.com/1424-8220/24/13/4233

[8] "Vehicle and Driver Monitoring System Using On-Board and Remote Sensors," *Sensors*, vol. 23, no. 2, 2023. [Online]. Available: https://www.mdpi.com/1424-8220/23/2/814

[9] "Smartwatch-Based Ecological Momentary Assessment for High-Temporal-Density, Longitudinal Measurement of Alcohol Use (AlcoWatch): Feasibility Evaluation," *JMIR Formative Research*, vol. 9, 2025. [Online]. Available: https://formative.jmir.org/2025/1/e63184/

[10] L. Lombardo, S. Grassini, M. Parvis, N. Donato, and A. Gullino, "Ethanol breath measuring system," in *2020 IEEE International Symposium*

*on Medical Measurements and Applications (MeMeA)*, 2020, pp. 1-6. doi: 10.1109/MEMEA49120.2020.9137215